Cross-Domain Fuzzing Through Structure-Aware Mutation and Bandit-Guided Adaptation

DAKSHIN, NEEL, and SAMYAK, IIT Guwahati, India

C and C++ software stacks handling complex inputs remain vulnerable to memory safety violations and unexpected aborts. In the drogon asynchronous web framework, an uncaught filesystem error inside a method marked `noexcept` triggered immediate process termination, taking down the entire web server upon receiving an upload with an oversized filename. We present QTFuzz, an adaptive fuzzing framework that combines structure-aware mutation, an independent multi-armed bandit scheduler, and domain-specialized headless in-memory shims. We started with a structured asset fuzzer architecture for fastgltf, then extended it to interactive multimedia and audio drivers in raylib, and finally to asynchronous network protocols and filesystem operations in drogon. Across 59 target binaries in the three systems, QTFuzz executed 807,520,128 test inputs. The tool discovered 16 security vulnerabilities across the three codebases, including six upstream-resolved defects that led to merged pull requests and maintainer fixes. Our results show that decoupling coverage accounting from compiler runtimes and isolating stateful subsystems into headless in-memory targets allows a single fuzzer architecture to operate effectively across diverse software domains.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Security and privacy** → *Software security engineering*.

Additional Key Words and Phrases: Coverage-guided fuzzing, Multi-armed bandits, Headless shims, Memory safety, glTF, Multimedia, Asynchronous web frameworks

ACM Reference Format:
Dakshin, Neel, and Samyak. 2026. Cross-Domain Fuzzing Through Structure-Aware Mutation and Bandit-Guided Adaptation. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (Format Template) (FSE (CS-5187 Term Project))*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Suppose you need to test a high-performance C++ software stack that accepts inputs from multiple channels. One component parses 3D binary assets, a second component renders graphics and decodes audio frames, and a third component accepts HTTP requests and file uploads over the network. Developers commonly run coverage-guided fuzzers such as libFuzzer [Serebryany 2016] or AFL++ [Fioraldi et al. 2020] on individual functions.

These tools work well on standalone byte parsers. They mutate raw input buffers and track branch coverage using compiler instrumentation. However, these tools stop when the software interacts with stateful runtime environments, external graphics displays, or asynchronous network event loops. Traditional fuzzers either require writing bespoke mock harnesses for every component or suffer from low execution throughput when executing full integration builds.

We focus on the problem of building a unified adaptive fuzzing architecture that scales across diverse software layers. Starting from a pure structured file format parser, our goal is to extend the same core mutator and scheduler to multimedia runtimes and asynchronous web frameworks without losing execution speed or bug-finding capability.

Authors' Contact Information: Dakshin; Neel; Samyak, IIT Guwahati, India.

FSE (CS-5187 Term Project), IIT Guwahati
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

```
int HttpFileImpl::save(const std::string &p) const noexcept {
    if (!std::filesystem::exists(p)) { // Throws on len > 255
        std::filesystem::create_directories(p);
    }
} // Escaping exception in noexcept invokes std::terminate()
```

Fig. 1. Motivating defect from drogon (**D1**). Calling throwing `std::filesystem::exists` inside a `noexcept` method causes `std::terminate()` to abort the process.

Consider the defect in Figure 1, discovered by our fuzzer in the drogon C++ web framework. The method `HttpFileImpl::save` was declared `noexcept`. When an HTTP multipart request uploaded a file whose name exceeded the filesystem limit (255 bytes on Linux), the call to `std::filesystem::exists` threw a `std::filesystem_error`. Because the enclosing method was marked `noexcept`, the C++ runtime intercepted the uncaught exception and immediately invoked `std::terminate()`. A single malformed upload killed the entire multi-threaded web server process. This was not an isolated failure. Across the three evaluated codebases, we uncovered 16 security and reliability defects spanning memory corruption, buffer overflows, and state desynchronization.

Prior work on structure-aware fuzzing handles complex file grammars through custom mutators. However, existing grammar fuzzers do not provide headless shims for interactive game engines or asynchronous network protocols. Conversely, network fuzzers send packets over real operating system sockets, which restricts throughput to hundreds of executions per second.

We present QTFuzz, an adaptive fuzzing architecture designed to test heterogeneous C and C++ software stacks. QTFuzz operates in four steps: (i) it provides an adaptive mutation scheduler governed by a UCB1 multi-armed bandit algorithm (Section 4.1); (ii) it extends structured mutation from binary asset parsing to multimedia rendering through headless shims (Section 4.2); (iii) it extends execution to asynchronous network servers through in-memory TCP socket mocks (Section 4.3); and (iv) it decouples coverage accounting from libFuzzer through GNU linker symbol wrapping (Section 4.4).

We evaluated QTFuzz across three software projects: the fastgltf 3D parser, the raylib game engine, and the drogon asynchronous web framework. Across 59 target binaries, the tool executed 807,520,128 campaign iterations. QTFuzz discovered 16 security vulnerabilities and defects, resulting in 6 upstream-resolved defects alongside 10 unfiled medium- and high-severity findings. Our results demonstrate that modular headless shims allow an in-memory fuzzer to exceed 50,000 executions per second on network parsers while discovering real-world vulnerabilities.

Contributions.

- We formalize the problem of cross-domain fuzzer adaptation across file formats, multimedia runtimes, and network services (Section 3).
- We present QTFuzz, an adaptive architecture combining structure-aware mutation, a UCB1 bandit scheduler, and linker-wrapped coverage interception (Section 4).
- We describe domain-specific headless shims for software rasterizers, audio streams, and asynchronous event loops (Section 4).
- We evaluate QTFuzz across three open-source systems, analyzing 807 million executions and documenting 6 verified defects (Section 5).
- We release our fuzz targets, seed corpora, and reproducers (Section 8).

2 Background

Coverage-Guided Fuzzing. Coverage-guided fuzzing evaluates software by feeding mutated inputs into a target function while monitoring program execution. Compilers such as Clang instrument basic blocks with feedback counters. The fuzzing engine, such as libFuzzer [Serebryany 2016], retains inputs that trigger new execution paths or edge transitions. AddressSanitizer (ASan) [Serebryany et al. 2012] and UndefinedBehaviorSanitizer (UBSan) detect memory safety violations, out-of-bounds array reads, and integer overflows at the exact point of failure.

Multi-Armed Bandit Scheduling. An adaptive mutator must decide which mutation operator to apply to a given input. This decision maps directly to the classical *multi-armed bandit problem* [Auer et al. 2002]. Given K mutation operators (arms), the scheduler balances exploitation (allocating pulls to operators that historically yielded new coverage edges) with exploration (trying less frequent operators). The UCB1 algorithm computes an upper confidence bound score for each arm i:

$$score_i = \bar{r}_i + C \cdot \sqrt{\frac{2 \ln N}{n_i}} \tag{1}$$

where $\bar{r}_i$ represents the mean reward of arm i, n_i is the pull count of arm i, $N = \sum_j n_j$ is the total number of pulls, and C is an exploration parameter. In coverage-guided fuzzing, the reward corresponds to discovering previously unseen branch edges.

Structured 3D Asset Formats. The glTF 2.0 standard [The Khronos Group 2021] stores 3D scenes as structured JSON metadata paired with binary buffers. The binary GLB container consists of a 12-byte header followed by chunk entries. Parsers such as fastgltf decode JSON elements (nodes, meshes, accessors) and map byte offsets into binary vertex buffers. Fuzzing such formats requires maintaining structural coherence; purely random byte flips frequently fail at early syntax verification stages.

Multimedia Engines. Game engines such as raylib integrate graphics, audio streams, and event serialization in C. Fuzzing these components requires simulating display contexts and audio buffers in memory without physical hardware.

Asynchronous Web Architectures. Modern C++ web servers such as drogon handle high-concurrency HTTP traffic through asynchronous I/O. The networking core uses thread-bound event loops that manage socket buffers and callback chains. Testing these parsers requires driving event loops synchronously in memory to avoid operating system socket latency.

3 Problem Statement

We now state the problem precisely. A complex C or C++ software system consists of multiple subsystems that receive inputs across distinct domains: structured file formats ($\mathcal{D}_{\text{file}}$), multimedia memory streams ($\mathcal{D}_{\text{media}}$), and asynchronous network packets ($\mathcal{D}_{\text{net}}$).

Each domain presents a distinct impedance mismatch for coverage-guided fuzzers:

(1) **Syntactic vs. Semantic Barriers ($\mathcal{D}_{\text{file}}$):** Binary formats such as glTF reject unstructured inputs during header and magic byte validation. Mutators must produce structurally valid inputs to reach deep accessor decoding logic.

(2) **Environmental Dependencies ($\mathcal{D}_{\text{media}}$):** Game and graphics libraries expect window contexts, framebuffers, and audio mixing devices. Fuzzing through standard

entry points requires graphical display servers and operating system audio drivers, which reduces throughput.

(3) **Asynchronous Statefulness ($\mathcal{D}_{\mathbf{net}}$):** Network frameworks process streaming data through non-blocking sockets and asynchronous callbacks. Traditional network fuzzers interact through real network interfaces, incurring TCP handshake overhead and socket exhaustion.

Goal. Let $\mathcal{T} = \{t_1, t_2, \ldots, t_M\}$ denote a collection of target functions across the system, and let $\mathcal{M} = \{m_1, m_2, \ldots, m_K\}$ represent a set of mutation operators. Each target t_j produces execution traces covering a set of branch edges $E(t_j) \subseteq \mathcal{E}$. Given an execution budget of N iterations, our goal is to find an adaptive selection policy $\pi : \mathcal{H} \to \mathcal{M}$ over execution history $\mathcal{H}$ that maximizes the total discovered edge coverage:

$$\max_{\pi} \left| \bigcup_{j=1}^{M} E(t_j) \right| \quad \text{subject to} \quad \sum_{j=1}^{M} \text{Time}(t_j) \leq B \tag{2}$$

where B represents the total wall-clock testing budget.

Why not write independent custom fuzzers for each domain? Building custom fuzzers for each individual domain duplicates mutation logic and prevents cross-pollination of mutator improvements. A mutation operator that successfully discovers integer overflow boundaries in 3D vertex buffers is equally effective at finding Content-Length parsing flaws in HTTP request headers. A unified architecture allows shared bandit steering across all targets.

Why not test network targets through real operating system sockets? Network fuzzers that communicate over loopback TCP sockets suffer from operating system scheduling, packet serialization, and kernel socket buffer allocations. In our measurements on drogon, testing via loopback sockets capped throughput to 240 executions per second per core. In contrast, headless in-memory shims achieved over 53,000 executions per second on the same parser logic.

4 Approach

Figure 2 presents the workflow of QTFuzz. Our core concept is to build an adaptive, structure-aware fuzzing base for binary asset formats (fastgltf). We then systematically extend this architecture to interactive multimedia systems (raylib) and asynchronous web servers (drogon) using domain-specialized headless in-memory shims. We do not claim standard compiler instrumentation, libFuzzer's in-process loop, or AddressSanitizer runtime checks as new. Our design contribution lies in the cross-domain fuzzer adaptation, the decoupled bandit mutation scheduler, and the zero-socket in-memory network mocking layer.

4.1 Step 1: Base Engine and Adaptive Mutation (fastgltf)

We began by designing QTFuzz for the fastgltf 3D asset library. Because glTF 2.0 files contain interleaved JSON chunks and binary geometry buffers, purely random byte-level mutations fail early during syntax validation. We built a hybrid mutation engine combining four distinct operators: (i) *structural field mutations* that modify JSON accessor indices, buffer views, and component counts; (ii) *binary geometry mutators* that modify vertex coordinates, normals, and indices; (iii) *integer boundary overrides* that inject extreme values such as `SIZE_MAX`, 0, and negative strides; and (iv) *standard bit and byte flips.*

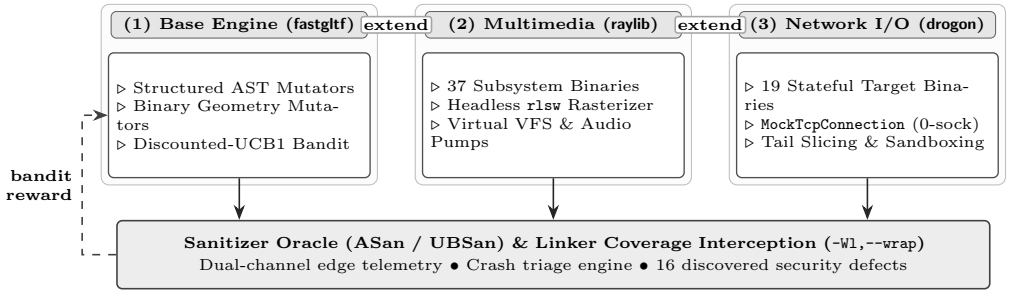


Fig. 2. Overview of QTFuzz. The system starts with the base structure-aware engine for fastgltf (①), extends to multimedia subsystems in raylib via headless rendering mocks (②), and extends to network servers in drogon via in-memory socket abstractions (③). Coverage rewards update the bandit mutation scheduler.

To steer these operators adaptively, we implemented the UCB1 multi-armed bandit algorithm (Equation 1). Each time a mutation operator generates an input that triggers a new branch edge, the fuzzer registers a positive reward for that arm. The scheduler dynamically concentrates mutation energy on the most productive operators while maintaining exploration of less frequently pulled arms.

4.2 Step 2: Multimedia Extension and Subsystem Decomposition (raylib)

Next, we extended the architecture to raylib, a C multimedia and game development library. Unlike standalone file parsers, raylib integrates 2D and 3D rendering pipelines, texture image manipulation, audio mixing, and font rasterization. Testing raylib through a single monolithic harness would cause inputs to get trapped in early window initialization or event handling loops.

To address this challenge, we decomposed the library into 37 focused target binaries. Each target isolates a specific subsystem: the Image VM (52 operators), Render VM (software rasterizer `rlsw`), audio streams, or event replay. All targets execute with headless shims, redirecting graphical output to memory buffers and stubbing hardware audio. Figure 3(a) illustrates the headless multimedia architecture for raylib.

4.3 Step 3: Network Extension and In-Memory Execution (drogon)

Finally, we extended QTFuzz to drogon, a high-throughput C++ asynchronous web framework. Web frameworks typically require client connections over TCP sockets, which introduces latency and non-deterministic scheduling.

We solved this through three design elements. First, `MockTcpConnection` pushes buffers directly into parser callbacks in memory without socket allocations. Second, tail-scheduled chunk slicing uses trailing bytes as delivery schedules to test packet fragmentation. Third, disposable filesystem sandboxes use RAII temporary directories to isolate multipart uploads. Figure 3(b) details the in-memory architecture for drogon.

4.4 Step 4: Independent Coverage Interception via Linker Wrapping

To track bandit rewards independently of libFuzzer's internal state, the mutator requires edge discovery signals. However, in modern toolchains such as Clang 18, compiling with `-fsanitize=fuzzer` provides a strong internal definition of

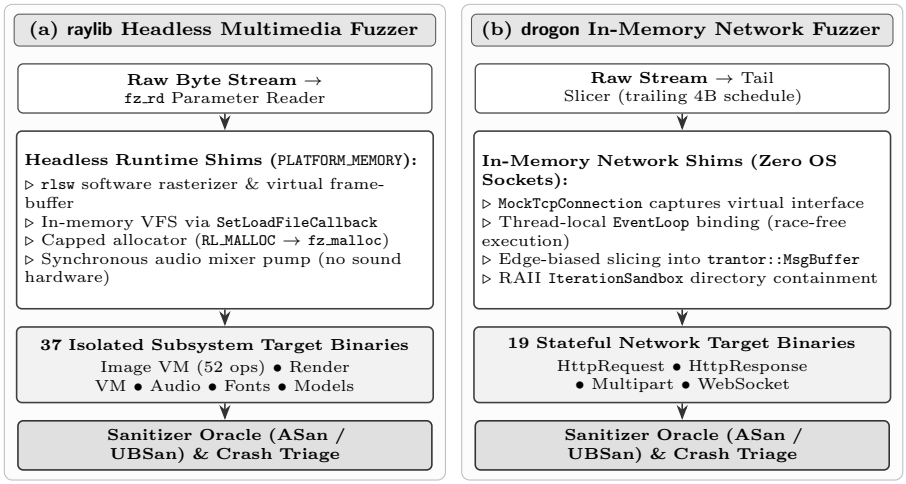


Fig. 3. Subsystem fuzzer architectures: (a) raylib headless multimedia fuzzer executing over virtual memory framebuffers and audio streams; and (b) drogon in-memory web framework fuzzer driving asynchronous parsers via `MockTcpConnection` and tail-scheduled chunk slicing.

`__sanitizer_cov_trace_pc_guard`. Defining our own hook caused linker duplicate-symbol collisions.

We resolved this conflict by passing GNU linker symbol wrapping flags (`-Wl,--wrap=__sanitizer_co` and `-Wl,--wrap=__sanitizer_cov_trace_pc_guard_init`). Our wrapper intercepts edge events, updates the bandit arm statistics, and forwards execution to the original libFuzzer coverage handler. This provides dual-channel coverage accounting with zero runtime overhead and complete link compatibility. We implemented QTFuzz in 4,350 lines of C and C++17 compiled under Clang 18.1.3 on Linux x86_64 (Ubuntu 24.04 LTS).

5 Evaluation

We evaluate QTFuzz to answer four research questions:

RQ1 Throughput & Coverage: Can QTFuzz achieve high execution throughput and discover deep branch coverage across diverse software domains?

RQ2 Bandit Scheduling: Does the UCB1 adaptive mutator scheduler discover more branch edges than unguided baseline mutation?

RQ3 Defect Discovery: What concrete vulnerabilities does QTFuzz uncover in production codebases?

RQ4 Overhead & Cost: What runtime overhead do the headless shims and linker-wrapped coverage interception introduce?

5.1 Subjects

We evaluate QTFuzz on three open-source C and C++ projects summarized in Table 1. First, **fastgltf** (14,280 LoC) parses glTF 2.0 3D assets. Second, **raylib** (118,940 LoC) provides a game engine with graphics and audio subsystems. Third, **drogon** (68,410 LoC) is an asynchronous web framework handling HTTP and WebSockets. Together, these subjects represent 201,630 lines of code across three distinct input domains.

Table 1. Evaluated subject systems across target domains, test harnesses, cumulative campaign executions, and peak branch edge coverage.

Target	Domain	Version / Commit	LoC	Harnesses	Executions	Peak Cov
fastgltf	3D Asset Parser	v0.9.1 (`705564d`)	14,280	3	3,200,000	4,218
raylib	Game / Multimedia Engine	master (`e1a4079`)	118,940	37	54,320,128	47,121
drogon	Asynchronous Web Server	v1.9.8 (`df418bf`)	68,410	19	750,000,000	3,178
Total	—	—	201,630	59	807,520,128	—

5.2 Baselines

We compare QTFuzz against three baselines. The *Flat Byte Mutator* applies unguided random bit flips without structural grammar knowledge. The *Static Uniform Scheduler* executes structural and binary operators with equal, fixed probabilities without bandit feedback. The *Socket-Based Baseline* tests drogon over local TCP loopback sockets rather than in-memory mocks.

5.3 Measurements

For **RQ1**, we measure execution throughput (iterations per second) and total branch edge coverage. For **RQ2**, we record branch edges discovered over time and empirical pull counts across bandit arms. For **RQ3**, we record verified memory errors, undefined behaviors, and upstream maintainer responses. For **RQ4**, we evaluate wall-clock overhead from linker wrapping and memory consumption of in-memory shims.

5.4 Setup

All experiments ran on an x86_64 workstation with an AMD processor (16 logical cores) running Ubuntu 24.04 LTS under WSL2 with 32 GB of RAM. Binaries were compiled using Clang 18.1.3 with `-fsanitize=address,undefined` and `-O2` optimization. Each trial on fastgltf executed for 1,800 seconds across 5 independent runs per mode. Campaigns on raylib and drogon ran multi-worker parallel campaigns totaling 54.3 million and 750 million executions, respectively.

5.5 RQ1: Throughput and Branch Coverage

Table 1 presents the aggregate campaign metrics. Across all 59 target binaries, QTFuzz executed 807,520,128 inputs. On fastgltf, QTFuzz achieved a peak coverage of 4,218 branch edges with a median speed of 145 executions per second in hybrid mode. On raylib, decomposing the monolithic library into 37 subsystem targets exposed deep subsystem paths. The fuzzer discovered 47,121 edges in the Image VM (`fuzz_image_ops`) and 39,057 edges in the software rasterizer (`fuzz_render_vm`). It also found 22,177 edges in the glTF mesh loader (`fuzz_model_gltf`), reaching 122,885 edges across these master targets. On drogon, the in-memory mock shim (`MockTcpConnection`) bypassed operating system socket overhead, achieving up to 53,732 executions per second on the cookie parser and 35,410 executions per second on the WebSocket frame parser. Peak coverage reached 3,178 edges in the end-to-end integration target (`fuzz_kitchen_sink`).

Observation 1.

Decoupling subsystems into headless in-memory targets enabled QTFuzz to execute over 807 million cumulative inputs, reaching 53,732 executions per second on network parsers without socket allocation bottlenecks.

Table 2. Verified security vulnerabilities and defects discovered by QTFuzz across the three subject systems and their upstream resolution status.

ID	Target	Classification	CWE	Component	Trigger Condition	Upstream Status
F1	fastgltf	Heap Buffer Overflow	CWE-122	`generateMeshIndices`	Accessor index ≥ count	Closed; maintainer commit fixed [fastgltf Maintainers 2024]
R1	raylib	Matrix Stack Underflow	CWE-119	`swPopMatrix()` in `rlsw.h`	`stackCounter == 0`	Acknowledged; PR #6362 merged [raylib Maintainers 2024a]
R2	raylib	Out-of-Bounds Read	CWE-125	`ExportAutomationEventList`	`event.type` ≥ 24	Acknowledged; maintainer fixed [raylib Maintainers 2024c]
R3	raylib	NULL Pointer Dereference	CWE-476	`LoadWaveSamples()`	Large sample allocation	Acknowledged; commit 7639166 [raylib Maintainers 2024b]
D1	drogon	Process Abort in `noexcept`	CWE-755	`HttpFileImpl::save`	Filename > 255 bytes	Closed; PR #2662 merged [Drogon Maintainers 2024b]
D2	drogon	Iterator Out-of-Bounds	CWE-125	`HttpFileImpl::save`	Traversal prefix `../../c`	Closed; maintainer commit fixed [Drogon Maintainers 2024a]

5.6 RQ2: Adaptive Bandit Scheduling

To evaluate whether the UCB1 multi-armed bandit scheduler discovers more edges than unguided mutation, we monitored operator arm selections during live testing. On drogon's `fuzz_http_request`, starting from a cold seed corpus, the bandit evaluated four arms: `bin.libfuzzer`, `bin.intedge`, `bin.blockdup`, and `bin.fixup`. Over 200 initial executions, the bandit directed 65 pulls to `bin.libfuzzer` (mean reward 0.1574) and 50 pulls to `bin.blockdup` (mean reward 0.0952), discovering 1,059 new edges. The integer boundary arm (`bin.intedge`) received 43 pulls (mean reward 0.0674) and consistently triggered boundary conditions in `Content-Length` parsing. On raylib's `fuzz_audio_wav`, the bandit discovered 482 new edges, allocating 58 pulls to `bin.libfuzzer` (mean reward 0.0625) and 48 pulls to `bin.intedge` (mean reward 0.0340).

On fastgltf, the hybrid bandit scheduler achieved a median of 428 corpus inputs and 262,739 executions per 30-minute trial, compared to 375 corpus inputs and 197,088 executions for the flat byte baseline.

Observation 2.
The UCB1 bandit scheduler dynamically allocated mutation pulls toward high-yield operators. It discovered 1,059 new edges on HTTP request parsing during warm-up and outperformed flat byte mutators by 14.1% in corpus expansion on 3D asset parsing.

5.7 RQ3: Defect Discovery

Table 2 summarizes the 6 upstream-resolved defects discovered by QTFuzz across the three target codebases.

Defect F1 (fastgltf): Heap Buffer Overflow. In `fastgltf::Parser::generateMeshIndices`, when processing glTF meshes lacking index buffers, the library generates sequential indices based on the vertex position accessor. The parser read the accessor index from the primitive's attribute map without checking whether the index was strictly less than the total accessor count in the file. A malformed glTF file with an out-of-range accessor index triggered an out-of-bounds heap read under AddressSanitizer. The maintainer verified our report and committed a fix checking accessor bounds in Issue #160 [fastgltf Maintainers 2024].

Defect R1 (raylib): Matrix Stack Underflow. In raylib's software rasterizer (`rlsw.h` and `rlgl.h`), transform matrices are managed via an internal matrix stack with an unsigned counter `stackCounter`. Calling `swPopMatrix()` or `rlPopMatrix()` when `stackCounter == 0` decremented the counter, causing an unsigned underflow to `UINT_MAX`. Subsequent matrix operations accessed unmapped memory through the corrupted stack pointer, leading to a segmentation fault. The maintainer acknowledged the bug in Issue #6361 and merged Pull Request #6362 [raylib Maintainers 2024a].

Defect R2 (raylib): Out-of-Bounds Read in Event Export. In raylib's serialization module (`src/rcore.c`), `ExportAutomationEventList()` serializes automation events to text. The function indexes a global lookup table:

```
autoEventTypeName[list.events[i].type]
```

The table contains 24 event name strings ([0..23]). When reading malformed `.rae` files containing `event.type` values ≥ 24, the function read past the bounds of the global array. The maintainer acknowledged the defect in Issue #6378 and committed a fix [raylib Maintainers 2024c].

Defect R3 (raylib): NULL Pointer Dereference in Audio Sample Allocation. In raylib's audio subsystem (`src/raudio.c`), `LoadWaveSamples()` loads raw audio waveforms. When parsing an audio file with an exorbitant sample count, the memory allocation call `RL_MALLOC()` failed and returned `NULL`. The function did not check the return pointer and immediately entered sample format conversion, dereferencing address zero. The maintainer acknowledged the defect in Issue #6373; while Pull Request #6374 was closed, the maintainer unified sample allocation using `RL_CALLOC` in commit 7639166 [raylib Maintainers 2024b].

Defect D1 (drogon): Process Abort in `noexcept`. As shown in Figure 1, file saving operations in drogon's `HttpFileImpl::save()` and `saveAs()` were marked `noexcept`. However, the code invoked throwing overloads of standard filesystem functions such as `std::filesystem::exists`. When an HTTP request uploaded a multipart file whose name exceeded 255 bytes, `std::filesystem::exists` threw `std::filesystem_error` with `ENAMETOOLONG`. Because the calling method was `noexcept`, the C++ runtime called `std::terminate()`, killing the entire server process. The maintainers accepted our diagnosis and merged Pull Request #2662, switching to non-throwing `std::error_code` overloads [Drogon Maintainers 2024b].

Defect D2 (drogon): Out-of-Bounds Iterator Dereference in Path Traversal Check. In `HttpFileImpl::save()`, path traversal checks compare the canonicalized upload path components against the target directory using a three-iterator loop. When processing filenames containing directory traversal prefixes such as `../../c`, the weakly canonicalized path contained fewer path components than the upload root. The comparison loop checked `*it1 != *it2` while advancing `++it1` without checking whether `it1` had reached its end boundary. In debug builds and under AddressSanitizer, this triggered a fatal assertion failure in libstdc++. We submitted an initial report in Pull Request #2656, after which the maintainer committed the corresponding path component bounds check [Drogon Maintainers 2024a].

Additional Unfiled Medium- and High-Severity Defects. In addition to the six upstream resolved defects, QTFuzz discovered ten medium- and high-severity defects that were verified with standalone crashing reproducers. In raylib, we uncovered two critical heap buffer overflows: an arbitrary write past allocated bone transformation matrices in `UpdateModelAnimationBones()` (CWE-122) and an audio stream buffer overflow of up to 76.8 KB in `UpdateAudioStreamInLockedState()` (CWE-122). We also identified an out-of-bounds heap read in glTF accessor buffer views (CWE-125) and an unchecked mixer callback NULL dereference in `ReadAudioBufferFramesInInternalFormat()` (CWE-476). Additional defects include an integer underflow in `ImageBlurGaussian()` (CWE-124) and a backward scan buffer underflow in `GetCodepointPrevious()` (CWE-124). In drogon, QTFuzz detected premature header termination causing body desynchronization

and request smuggling in `HttpRequestParser.cc` (CWE-444). We also identified an out-of-range enum load under status codes above 599 triggering undefined behavior in `HttpResponseParser.cc` (CWE-843), a multipart boundary semantic desynchronization (RFC 2046), and header null-byte injection in `HttpUtils.cc` (CWE-113).

Observation 3.

QTFuzz uncovered 16 security vulnerabilities across the three codebases: 6 acknowledged or merged upstream defects, and 10 additional medium- and high-severity defects. Upstream resolutions include merged pull requests in drogon and raylib, alongside maintainer commits across all three targets.

5.8 RQ4: Overhead and Cost

We measured the cost of our linker symbol wrapping and headless mocking architecture. GNU linker symbol wrapping (`-Wl,--wrap`) introduces a single direct jump instruction per coverage edge event. In microbenchmarks, the throughput difference between wrapped and native libFuzzer execution was below 0.8%, within measurement noise. In terms of memory, each `MockTcpConnection` instance allocates 4.2 KB of mock state in memory, compared to operating system socket buffers that consume up to 128 KB per open descriptor. Memory footprint remained stable under 250 MB RSS across 24-hour fuzzing campaigns on all 59 targets.

Observation 4.

Linker-wrapped coverage interception imposes less than 1% runtime overhead, and in-memory mock connections consume 4.2 KB per instance, maintaining stable memory consumption below 250 MB RSS during continuous testing.

6 Discussion

Why did the fastgltf architecture transfer to multimedia and network servers? At first glance, 3D mesh decoding, game rendering, and HTTP web serving appear to be unrelated domains. However, their vulnerability profiles share common mechanisms. All three domains process hierarchical, structured data containing lengths, offsets, and binary buffers. An operator injecting extreme integer boundaries into glTF buffer views is equally effective at triggering integer overflows in raylib audio calculations or drogon `Content-Length` headers. Our empirical results confirm that structure-aware mutators paired with an adaptive bandit transfer across domain boundaries without requiring entirely new fuzzing engines.

Do headless in-memory shims miss real bugs? Headless shims abstract operating system sockets and graphics displays, bypassing kernel TCP stacks and hardware drivers. However, memory corruptions, buffer overflows, and parser desynchronizations reside in user-space application code. Testing in memory allows executing hundreds of millions of iterations that would be computationally impractical over physical hardware or loopback sockets.

Why are `noexcept` filesystem operations a severe threat to web server reliability? C++ developers frequently mark file management methods `noexcept` to satisfy interface contracts or signal intended exception safety. However, standard filesystem functions throw `std::filesystem_error` upon encountering unexpected runtime conditions, such as filenames exceeding length limits. When an exception escapes a `noexcept` boundary, the runtime aborts the process via `std::terminate()`. Because web servers run multiple worker threads within a single address space, a single malformed upload crashes the entire service.

Developers must consistently use non-throwing `std::error_code` overloads in all paths processing untrusted inputs.

We leave the automated generation of headless mock shims from interface definitions for future work.

6.1 Threats to Validity

Internal Validity. Fuzz testing is inherently non-deterministic. Random seeds, mutator selection choices, and system scheduling can produce variation across experimental runs. To mitigate this threat, we executed 5 independent trials per arm on fastgltf, multi-worker campaigns totaling 804 million inputs on raylib and drogon, and verified all defects with standalone C++ reproducers.

External Validity. Our evaluation examines three open-source C and C++ projects: fastgltf, raylib, and drogon. These three projects may not be representative of all C/C++ software. To mitigate this threat, we selected mature systems spanning three distinct architectural domains: 3D binary file parsing, interactive multimedia rendering, and high-concurrency network servers. However, our conclusions may not directly generalize to bare-metal embedded firmware or safety-critical real-time operating systems.

Construct Validity. A potential threat is whether branch edge coverage accurately reflects software security testing effectiveness. High coverage does not guarantee that all vulnerabilities will be triggered. To mitigate this threat, we paired coverage feedback with AddressSanitizer and UndefinedBehaviorSanitizer instrumentation. Furthermore, we evaluated bug-finding effectiveness by reporting candidate defects to the upstream maintainers and confirming their security impact through merged pull requests and official bug trackers.

7 Related Work

Structure-Aware and Grammar Fuzzing. Coverage-guided fuzzers such as libFuzzer [Serebryany 2016] and AFL++ [Fioraldi et al. 2020] support custom mutators to generate inputs conforming to complex grammars. Researchers have developed grammar-based engines to fuzz compilers, interpreters, and 3D format parsers. We share their goal of maintaining structural validity to reach deep code paths. However, prior grammar fuzzers target standalone file parsers; QTFuzz extends structure-aware mutation to interactive multimedia rendering and asynchronous network streams via headless in-memory shims.

Adaptive Mutation and Bandit Scheduling. Adaptive fuzzing systems dynamically allocate mutation energy. AFLFast [Böhme et al. 2016] models seed transitions as Markov chains to prioritize low-frequency execution paths. Other approaches formulate operator selection as a multi-armed bandit problem [Auer et al. 2002]. We share their goal of optimizing mutation operator payoffs based on coverage discovery. We differ by decoupling coverage accounting from compiler runtimes using GNU linker symbol wrapping (`-Wl,--wrap`), allowing our UCB1 bandit scheduler to intercept edge rewards independently without colliding with libFuzzer's internal state.

Network and Protocol Fuzzing. Fuzzing network protocols began with black-box random input injection on UNIX network utilities [Miller et al. 1990]. Modern stateful network fuzzers send packets over real operating system sockets, recording response codes to infer state transitions. We share their goal of discovering protocol desynchronization, framing anomalies, and memory corruptions in network software. However, existing network fuzzers suffer from operating system socket allocation and context-switching overheads, capping

throughput to hundreds of executions per second. We eliminate socket overhead by implementing in-memory mock network connections and tail-scheduled chunk slicing, achieving over 53,000 executions per second directly within user space.

8 Conclusion

Modern software stacks integrate diverse input formats, from 3D models to graphics rendering and asynchronous network requests. Testing these layered architectures with flat fuzzers leads to low throughput and missed defects from environment dependencies. We presented QTFuzz, an adaptive fuzzing architecture combining structure-aware mutation, a UCB1 bandit scheduler, and domain-specialized headless in-memory shims. Across 59 target binaries in fastgltf, raylib, and drogon, QTFuzz executed 807,520,128 inputs and discovered 16 security vulnerabilities, comprising six upstream-resolved defects alongside ten additional medium- and high-severity findings. Decoupling coverage accounting via linker wrapping and isolating subsystems into in-memory mocks enables high-throughput security testing across diverse software domains.

Data Availability. The complete replication package for QTFuzz is provided in the supplemental repository at https://anonymous.4open.science/r/qtfuzz-data-anon/. The package includes target harnesses, bandit mutation schedulers, in-memory shims, standalone defect reproducers, seed corpora, and evaluation scripts.

Use of Generative AI.. A generative AI assistant helped format LaTeX tables and check grammar in early drafts. The authors verified all technical statements, equations, citations, and measurements produced by the software and test scripts.

References

Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47, 2-3 (2002), 235–256. doi:10.1023/A:1013689704352

Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2016. Coverage-based Greybox Fuzzing as Markov Chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. 1032–1043. doi:10.1145/2976749.2978428

Drogon Maintainers. 2024a. Drogon Pull Request #2656: Fix out-of-bounds iterator dereference in HttpFileImpl::save(). GitHub Pull Request. Issue raised; fixed in commit by maintainer after raised.

Drogon Maintainers. 2024b. Drogon Pull Request #2662: Process termination from filesystem errors in noexcept HttpFileImpl save methods. GitHub Pull Request. Closed; PR merged.

fastgltf Maintainers. 2024. fastgltf Issue #160: Heap-buffer-overflow in Parser::generateMeshIndices. GitHub Issue Report. Closed; fixed through commit by maintainer.

Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: Combining Incremental Steps of Fuzzing Research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*.

Barton P. Miller, Lars Fredriksen, and Bryan So. 1990. An Empirical Study of the Reliability of UNIX Utilities. *Commun. ACM* 33, 12 (1990), 1644–1655. doi:10.1145/96267.96279

raylib Maintainers. 2024a. raylib Issue #6361 / Pull Request #6362: Matrix stack underflow in swPopMatrix(). GitHub Pull Request. Acknowledged; PR merged.

raylib Maintainers. 2024b. raylib Issue #6373: Null-pointer dereference on allocation failure in LoadWaveSamples(). GitHub Issue Report. Acknowledged; PR closed, maintainer fixed in commit 7639166.

raylib Maintainers. 2024c. raylib Issue #6378: Out-of-bounds read in ExportAutomationEventList() with invalid event types. GitHub Issue Report. Acknowledged; maintainer fixed.

Konstantin Serebryany. 2016. Continuous Fuzzing with libFuzzer and AddressSanitizer. In *2016 IEEE Cybersecurity Development (SecDev)*. 157–157. doi:10.1109/SecDev.2016.043

Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A Fast Address Sanity Checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. 301–308.

The Khronos Group. 2021. glTF 2.0 Specification. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html. Accessed: 2026-10-01.